LockBox

Course Code: CSE 3200

Course Title: Software Development Project ||

Project Mentor: Mst. Sahela Rahman

Team Members:

- Md Samiul Islam -> ID: 0322320105101001
- Simanto Sarkar -> ID: 0322320105101003
- Chanchal Kumar -> ID: 0322320105101055

Batch: 23rd
Semester: 6th

Dept: Computer Science & Engineering

Date: 28 February, 2026

Topic to be discussed

- Introduction
- Objective
- Existing Work
- Proposed Methodology & Tools
- Architecture Diagram, Logic Flowchart & ER Diagram
- Project's Expected Result
- Project Tentative timeline
- Conclusion and Future work
- References

Introduction

As digital footprints expand, the average user manages dozens of unique credentials. This has led to two dangerous behaviors: "password recycling" or reliance on centralized cloud-based managers. While convenient, cloud managers represent a single point of failure and an attractive target for sophisticated state-level and independent hackers. LockBox is a desktop-based "Zero-Knowledge" utility designed to eliminate these risks by ensuring that sensitive data never leaves the user's local hardware.

Objective

- Hardened Encryption: To implement a "double-wall" security layer using PBKDF2 for key derivation and AES-256-GCM for data encryption.

- Data Sovereignty: To utilize SQLite as a localized, serverless database engine, ensuring the user has total physical control over their vault file.

- Zero-Knowledge Architecture: To ensure that the application itself never knows the Master Password; it only facilitates the math required to unlock the data.

- Modern UI/UX: To provide a streamlined, "Flat Design" interface that makes security accessible rather than cumbersome.

Existing Work

Before designing LockBox, we analyzed industry leaders to identify security gaps and best practices:

- **KeePass (Local-Only)**: The gold standard for local password management. It uses the .kdbx file format. While highly secure, its UI is often cited as dated and complex for average users.

- **Bitwarden (Open Source)**: Known for its transparent security audits. It uses PBKDF2 with 600,000+ iterations for key derivation. LockBox adopts this standard for local deployment.

- **LastPass (Cloud-Based)**: Suffered major breaches in 2022/2023. This reinforces the need for local-only managers where the user—not the provider—holds the encrypted vault.

Key Findings: The "Master Password" must never be stored. Instead, a derived key must be used for encryption, and a high-iteration hash should be used for authentication to prevent brute-force attacks on modern GPUs.

Proposed Methodology

1. Key Derivation: PBKDF2 with HMAC-SHA256

The Master Password is not used directly. Instead, we use PBKDF2 (Password-Based Key Derivation Function 2):

- Salt: A unique 16-byte random salt stored in the DB.
- Iterations: 600,000 (minimum).
- Derived Key: 256 bits.
- Purpose: This makes "offline" brute-force attacks extremely slow and expensive for hackers.

Proposed Methodology

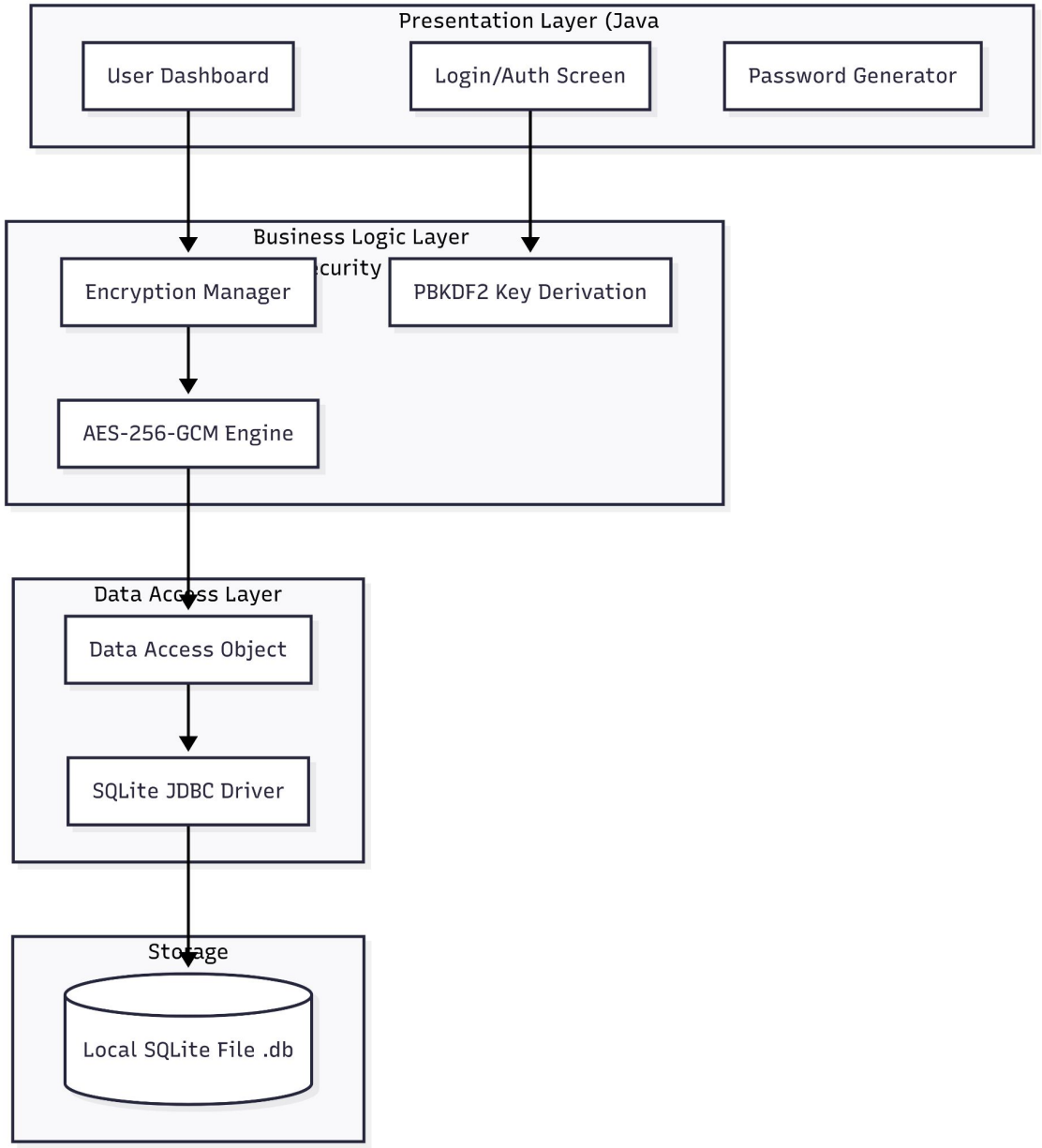
2. Encryption: AES-256-GCM

While many apps use AES-CBC, LockBox uses AES-GCM (Galois/Counter Mode).

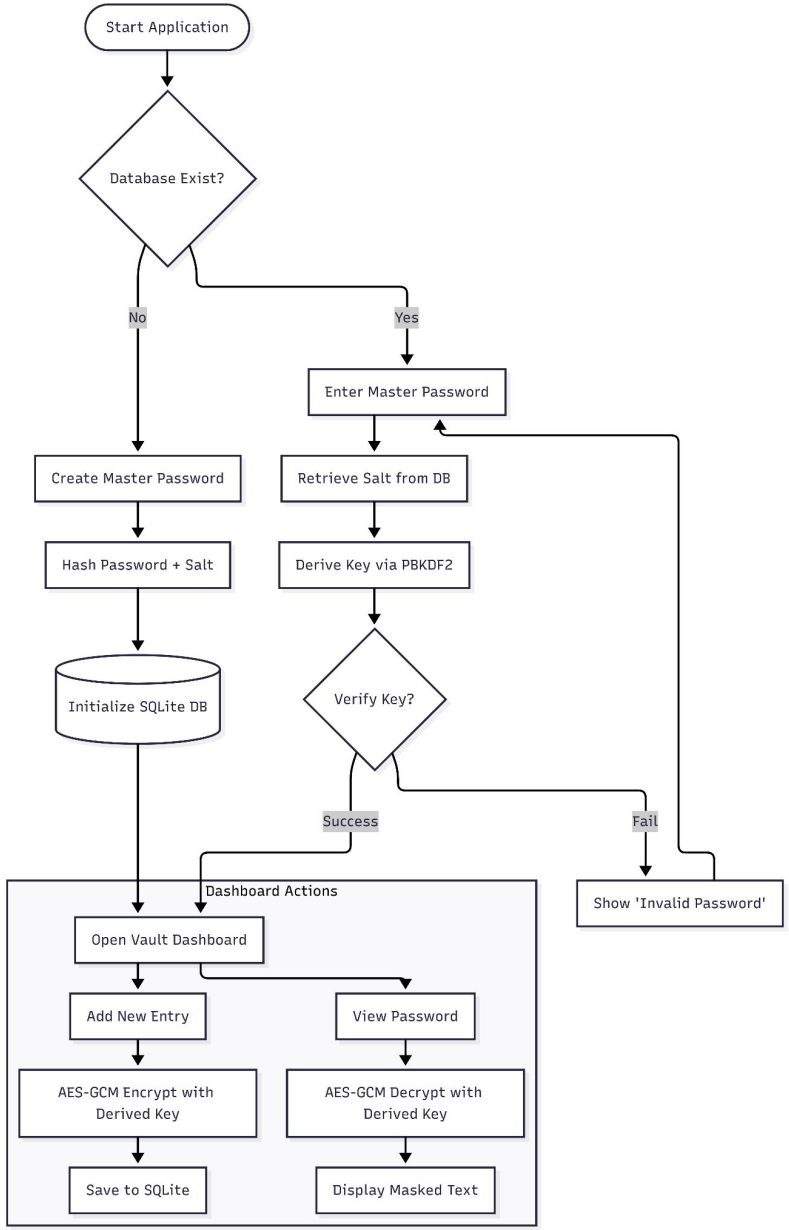
- Why GCM? Unlike CBC, GCM provides Authenticated Encryption. It ensures that the data has not been tampered with by adding an Authentication Tag to the encrypted block.

- IV (Initialization Vector): A unique 12-byte IV is generated for every single password entry to prevent pattern recognition.

Proposed: Architecture Diagram



Proposed: Logic Flowchart and ER Diagram



VAULT		
int	id	PK
text	site_name	
text	username	
blob	password_blob	
blob	iv	

Project's Expected Result

- **A Standalone Executable**: A JAR or native installer for the LockBox application.

- **Encrypted Local Vault**: A .db file that is cryptographically secure even if stolen.

- **Security Documentation**: A guide explaining the implementation of the AES-GCM and PBKDF2 parameters.

- **Feature-Complete UI**: A functional dashboard allowing for full CRUD (Create, Read, Update, Delete) operations on user credentials with automated "Clip-and-Clear" clipboard safety.

Conclusion and Future Work

Conclusion (The Now)	Future Work (The Next)
✓ Local-First Security Architecture	→ Browser Extension for Auto-fill
✓ Military-Grade AES-256-GCM Encryption	→ Biometric (Fingerprint) Unlocking
✓ 100% User Data Sovereignty	→ Mobile View-Only Companion App
✓ Zero-Knowledge Logic Flow	→ Encrypted Cloud Backup Support

References

- NIST Special Publication 800-63B: Digital Identity Guidelines regarding password complexity and storage. [Original Source: nist.gov]
- OWASP Password Storage Cheat Sheet: Recommendations for PBKDF2 iterations and salt lengths. [Original Source: owasp.org]
- SQLite Documentation: For local storage implementation. [Original Source: sqlite.org]
- Java Cryptography Architecture (JCA) Reference Guide: Implementation of Cipher and SecretKeyFactory. [Original Source: oracle.com]